

Beyond the Editing Canvas: Evidence Divergence in OOXML-to-LLM Ingestion

Side Liu¹ Jiangpeng Liu¹ Jinwen Xin² Guojun Peng² Jiang Ming¹
¹Tulane University ²Wuhan University

Abstract

LLM pipelines increasingly ingest Office Open XML (OOXML) documents (Word, Excel, and PowerPoint files) as first-class evidence in financial, compliance, and retrieval-augmented workflows, implicitly assuming semantic integrity: that the evidence consumed by the model matches the content shown in the Microsoft Office suite editing canvas. We show that this assumption can fail in OOXML-to-LLM pipelines. The same specification-valid OOXML file can yield one evidentiary view in Microsoft Office and another when extracted for an LLM. Each view is treated as authoritative by its consumer, a condition we call *plural ground truth*. The ingestion contract rarely states which view and semantic roles become model evidence or preserves how that evidence was derived. We call the specification-grounded OOXML constructions that induce such divergence *evidence forks*.

We systematically traverse and mine the OOXML specification and confirm 21 evidence forks across Excel, Word, and PowerPoint, spanning six dimensions of view construction. All 13 tools in our extraction panel emit evidence from at least one fork. We test four native-ingestion LLM APIs and seven web chatbots. Each test document carries a *trap*: a task-relevant fact exposed by extraction but not shown in Office. Across this 21-mechanism evaluation, the four APIs return the trap in 48–76% of trials. For 20 of 21 mechanisms, at least one of the eleven interfaces returns the trap. Our measurements further show that exposure is shaped upstream of the model by the ingestion path and extractor configuration. A source-level survey of sixteen popular open-source LLM projects further shows that default OOXML ingestion paths concentrate on affected extractor families.

1 Introduction

Word, Excel, and PowerPoint files (.docx, .xlsx, and .pptx) are ubiquitous [1] and increasingly serve as inputs to large language model (LLM) applications [2]. Retrieval-augmented generation (RAG) systems load them into searchable knowledge bases [3, 4], while financial agents use spreadsheets and

reports to support analysis and decisions [5, 6]. These workflows rely on an *ingestion* step that users do not see: a back-end parser, converter, or loader turns an Office Open XML (OOXML) file into text for the model. They therefore assume *semantic integrity*: the model receives the same content that the user reviewed in Microsoft Office’s default editing canvas. When this assumption fails, the opaque ingestion layer becomes a document supply-chain vulnerability.

Prior work on PDF documents has studied render/extract divergence, showing that content masking can make online services act on evidence that readers do not see [7]. Recent work on PDF-to-LLM pipelines examines how content masking, phantom tokens, glyph remapping, reading order, and layout shape the evidence delivered to LLMs [8–10]. OOXML exposes a different, more structural surface. It encodes values together with formulas, cached results, formatting rules, application state, metadata, and compatibility markup [11, 12]. For example, an .xlsx cell can carry a formula, its cached result, and a number format that changes what Excel displays. OOXML defines the roles of these representations, but an OOXML-to-LLM ingestion contract rarely specifies whether model evidence should reflect the recalculated, cached, stored, or displayed value. What a user sees in Microsoft Excel can therefore differ from what an LLM receives after extraction.

OOXML document security research has exposed dangerous specification features [13], signature and rendering gaps [14], ZIP-parser differentials [15], and hidden content in package structure and revision data [16]. Recent LLM-facing studies have also tested hidden and off-page Office content against loaders, grading systems, and web-assisted review workflows [17–19]. Together, these studies establish that selected Office carriers can survive ingestion and affect downstream tasks. Their methodological starting point is a predefined set of document manipulations, whose effects they then measure in a target workflow.

In this paper, we start from the OOXML specification and systematically study how *plural ground truth* can arise from a single specification-valid OOXML file: Microsoft Office’s default editing canvas and an ingestion pipeline expose in-

compatible task-relevant evidence. Each consumer treats its own view as authoritative; this operational plurality does not imply normative equivalence under OOXML. This violates the semantic-integrity assumption that a model reasons over the evidence shown in the review view. We call a specification-grounded OOXML construction that induces this evidentiary divergence an *evidence fork*. The fork arises during view construction: the stored package is unchanged, but Office and the ingestion path promote different content to task evidence.

In all, we confirm 21 evidence forks across Word, Excel, and PowerPoint, spanning representation, state, compatibility, visibility, scope, and linearization. Across 13 OOXML extraction tools, mechanism reach is sharply bimodal: seven mechanisms reach 11–33% of applicable tools and fourteen reach 67–100%, with none in between. The split follows where the trap resides in the OOXML package: ordinary content is broadly exposed, whereas attributes, relationships, and alternate branches are tool-specific. We build ten realistic, specification-valid Office documents per mechanism from TAT-QA financial-report excerpts [20], 210 in total, and test them against four native-ingestion LLM APIs and seven mainstream web chatbots. Each instance carries a *trap*: a task-relevant fact exposed by extraction but not shown in Office. Across 8,400 API trials, models return the trap in 48–76% of runs, depending on the API. At least one interface returns the trap for 20 of 21 mechanisms. Exposure varies with the ingestion path even across interfaces that a provider identifies as using the same model. A service’s exposure pattern can also match it to candidate ingestion configurations that behave identically on our probes. A source-level survey of sixteen popular open-source LLM projects shows that their default OOXML ingestion paths concentrate on affected extractor families.

To our knowledge, this is the first systematic, specification-grounded measurement of divergence between the default Office view and the task evidence delivered by OOXML-to-LLM pipelines. Our study covers Word, Excel, and PowerPoint across heterogeneous extractors and LLM interfaces. We make three contributions:

- We introduce a specification-guided method for discovering and confirming evidence forks, yielding a unified cross-format catalog and a six-dimensional taxonomy of OOXML view constructions.
- We conduct a large-scale measurement of how evidence forks propagate through extraction tools and LLM interfaces. We find broad exposure across the tested systems and show that it is shaped by ingestion path and extractor configuration, not by model choice alone.
- We assess ecosystem reach and detectability across open-source LLM projects and ordinary documents, revealing shared extractor dependencies and the limits of presence-only screening for common OOXML channels.

2 Background

This section provides the document-format context for our study. We first describe how OOXML packages organize document content and supporting state, then use a formula-cache example to illustrate how the same file can yield different evidence across consumers. We next introduce the Office applications and extraction tools that construct these views. Finally, we position our work relative to prior research on Office-document security and document attacks on LLMs.

2.1 OOXML Documents

OOXML Structure Office Open XML (OOXML) is the file format of the common Office documents, mainly .docx, .xlsx, and .pptx. Each is an Open Packaging Conventions (OPC) container: a ZIP archive of typed *parts* (XML documents and media) tied together by *relationships* [11, 12]. Listing 1 shows the internal structure of a real Word .docx: a main story, supporting parts, and the relationships that connect them. The visible text is the main story in word/document.xml, while its styles, numbering, fonts, and recalculation and compatibility settings live in sibling parts; package-level parts such as [Content_Types].xml and _rels/.rels declare each part’s type and record the relationships. Beyond these, a document may carry further content such as comments, footnotes, headers and footers, and drawing objects, each in a part that a given consumer may or may not traverse. User-facing applications resolve these parts through rendering, recalculation, style application, and data binding. Text extraction tools, by contrast, linearize the package through partial parsers that prioritize speed, portability, or text coverage over faithful application-level resolution.

```
example.docx
|-- [Content_Types].xml    %content-type map
|-- _rels/.rels            %package relationships
|-- docProps/              %core.xml, app.xml, ...
|-- customXml/             %item1.xml (data-bound)
'-- word/
    |-- document.xml        %main story (body text)
    |-- styles.xml          %styles + number formats
    |-- settings.xml        %recalc + compatibility
    |-- theme/, numbering.xml, fontTable.xml ...
    '-- _rels/document.xml.rels

word/document.xml:
<w:body>
  <w:p><w:r><w:t>Topic 606 ...</w:t></w:r></w:p>
  ...
</w:body>
```

Listing 1: Inside a real .docx: the OPC package of typed parts, with the word/document.xml body story.

Motivation Example The same object in an OOXML file can carry multiple representations with different semantic roles,

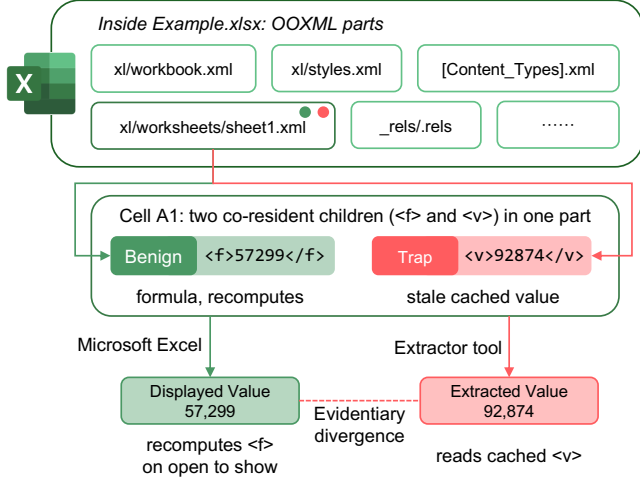


Figure 1: A formula/cache evidence fork: Microsoft Excel shows the benign value 57,299 while an extractor feeds the trap value 92,874 to the model.

and an Office editor and an extractor can expose different ones. Figure 1 shows the simplest case. A worksheet cell stores a formula `<f>` next to its cached result `<v>`: Microsoft Excel recomputes the formula on open and shows the benign value 57,299, while an extractor without a formula engine reads the attacker-planted cache `<v>` of 92,874. Neither reading is hidden or malformed, yet the two consumers disagree.

OOXML permits the formula and its cached result to co-exist and assigns them distinct roles. Excel recalculates and renders the formula result, whereas a formula-less extractor may expose the cache directly. The ingestion contract does not state which of these representations becomes model evidence or record that the emitted value came from a cache. The specification-valid file opens cleanly and shows the benign 57,299 in Excel’s default editing canvas, with no macro to run and no part to repair. An attacker who controls the cell can make the model ingest the trap 92,874 from the extractor while the analyst reviews the benign value. This formula/-cache construction is a representation-level evidence fork; the difference between the two outputs is the resulting evidentiary divergence. The remaining classes arise from other decisions involved in constructing a document view.

2.2 OOXML Consumers

Office Applications A user opens and reviews an Office document in an office application that renders it, chiefly the Microsoft Office suite [21] (Word, Excel, PowerPoint) or LibreOffice [22]. Such an application recalculates formulas, applies number formats and styles, refreshes data bindings, resolves compatibility branches, and hides filtered or hidden content. We therefore choose the Microsoft Office suite’s

default editing canvas as the human-facing oracle in our threat model, reflecting a widely deployed OOXML implementation and the review workflow we study.

Extraction Tools When the same document reaches an LLM, its text is produced not by such an application but by a separate extraction tool that runs first. These tools are libraries: format-specific parsers such as `python-docx` [23], `openpyxl` [24], and `python-pptx` [25]; converters such as `docx2txt` [26], `xlsx2csv` [27], `pandoc` [28], Apache Tika [29], and LibreOffice [22]; and meta-extractors such as `markdown` [30], `Unstructured` [31], and `Docling` [32] that wrap the others. Each is a *partial* parser that linearizes the package for speed, portability, or text coverage rather than reproducing an application’s resolution. Across mainstream open-source RAG and agent projects (LangChain, LlamaIndex, Haystack, AutoGen, Unstructured, Docling, markdown, RAGFlow, Dify, Onyx, and txtai, among others), default OOXML ingestion paths concentrate on the same small set of leaf extractors, making loader choice an indicator of potential exposure.

2.3 Related Work

Office-Document Security Müller et al. [13] systematically read the OOXML and ODF specifications to catalog dangerous-by-design features and application-level content masking. Other work studies signature and rendering gaps [14], container and parser differentials [15], hidden data in package structure and revision state [16], and executable macro semantics [33]. These studies target Office applications, hosts, signatures, or forensic recovery. We instead study the evidence supplied to LLMs; our fixtures are specification-valid and open without repair, and malformed containers and macros are outside our threat model.

Document Attacks on LLMs Recent PDF-to-LLM attacks use phantom tokens, glyph remapping, reading-order manipulation, and layout to make models act on evidence that differs from a page’s rendered content [8–10]. The closest LLM-facing Office document studies examine selected hidden and off-page carriers in Word documents. Castagnaro et al. [17] evaluate six techniques applicable to DOCX against loaders and RAG systems. Sili et al. [18] test four predefined DOCX injection techniques in LLM-based grading. Tomazini et al. [19] use a single white-font Word carrier to alter web-LLM review recommendations. These studies establish carrier-level and task-level effects in DOCX, but do not examine Excel or PowerPoint. We instead derive 21 OOXML view constructions from the specification across Word, Excel, and PowerPoint and trace them through heterogeneous extraction tools and LLM interfaces. Prompt-in-content and indirect-prompt-injection studies place attacker instructions in documents or other structured inputs to redirect model behavior [34–36]. Other work uses LLMs to detect malicious OOXML files [37], develops detectors for hidden prompts [38], and measures real-world prompt injection in resume-screening workflows [39].

2.4 Threat Model

We consider an OOXML-to-LLM workflow in which a user reviews a document in Microsoft Office’s default editing canvas, while an ingestion pipeline independently extracts evidence from the same file for factual question answering or summarization. The attacker supplies the document and controls its OOXML content and package structure, but not the extractor, model, or prompt. The file must be specification-valid, open in Word, Excel, or PowerPoint without repair, display the benign content, and not display the trap in the default editing canvas. We exclude malformed containers, macros, and executable payloads. The attack succeeds when the LLM returns attacker-chosen, task-relevant content that is absent from the Office review view, either by substituting a queried fact or adding a fabricated disclosure. We model ordinary review in the default editing canvas, not forensic inspection through auxiliary views or package internals.

3 Workflow

We identify evidence forks using the workflow in Figure 2. First, we turn the OOXML specification into a queryable knowledge base (step ①), then traverse it with an LLM to produce candidate records (②). We attempt to instantiate each record as a minimal Office fixture (③) and test it with the Office oracle (④) and extraction panel (⑤). Finally, we deduplicate passing records into mechanisms and apply scope screening (⑥), yielding the confirmed catalog.

3.1 Knowledge Base and Mining

The OOXML specifications combine machine-readable schemas with extensive prose. We organize both into a queryable knowledge base (①) and mine it with an LLM agent to produce candidate records (②).

Knowledge Base Construction The OOXML specification comes in two forms. The *prose* reference—ECMA-376 [11, 12] and the Microsoft extension documents [40–42], which describe each element in numbered clauses and states the rules a parser must obey. The *XSD* (XML Schema Definition) schemas [43] give the same elements a machine-readable grammar: what each element may contain, its attributes, and their multiplicities. We build one index from each. Parsing the schemas yields 15,884 schema records, one per element or type, each carrying its content model, `xs:choice` siblings, occurrence, defaults, and references to its associated prose clauses. Cutting the prose at its clause headers yields 5,206 prose sections, each anchored to an element name and assigned an identifier that preserves its specification source and section number. Both are emitted as JSON. Schema records store these source-qualified prose-section identifiers in `prose_refs`, keeping same-named clauses distinct.

Mining We traverse the knowledge base with an LLM (Claude Opus 4.8). The base is too large for one context, and per-element attention degrades in very long inputs, so we partition it into element-anchored chunks of at most 500K characters that never split a section—23 chunks across the eight blocks: the ECMA-376 and the Microsoft extensions. Each chunk lists its elements as `[name] title ($section)` followed by the element’s full clause—its attribute table and any precedence, selection, or visibility rule. For every chunk we issue one agent call that goes element by element and, whenever an OOXML construct could cause Office rendering and text extraction to expose different task-relevant content, emits a candidate record with the following fields:

```
(element, $section, format, quote, render,
extract, strength)
```

Each field feeds a later stage. `element` and `$section` anchor the record to a construct, support anchor-level consolidation, and locate it within the traversed specification blocks; `format` selects the document type to instantiate; `quote` is the verbatim specification sentence that defines the construct, its semantic role, or its processing rule, providing an audit trail; `render` and `extract` describe the hypothesized Office and extractor outputs, which become the BENIGN and TRAP tokens during empirical instantiation (③); `strength` records how directly the specification supports the record. Each call writes its tuples to a result file. A deterministic parser merges the 23 result files and consolidates records sharing the same specification anchor on `(element, $section)`, yielding 639 specification-anchored candidate records. At this stage, each record is an observation tied to one specification location, not yet a distinct mechanism.

3.2 Instantiation and Confirmation

The candidate records are not yet confirmed evidence forks: the specification establishes the construct and its semantics, but not how an Office application or extractor operationalizes it. Confirmation therefore uses two behavioral tests: the default Office editing canvas must show BENIGN rather than TRAP, and at least one extractor must emit TRAP. We attempt to instantiate all 639 candidate records as minimal, specification-valid Office fixtures (③) and evaluate them with the Office oracle (④) and extraction panel (⑤). Passing records then undergo mechanism-level deduplication and scope screening (⑥) to form the confirmed catalog.

Instantiation To keep instantiation auditable, we ask an LLM to generate a short builder program rather than package bytes directly. Using a builder makes every package edit explicit and reproducible. It also lets us distinguish construction failures from mechanism failures before applying the two behavioral gates. We prompt the model with its supporting candidate record from the mining stage: the element and section, the

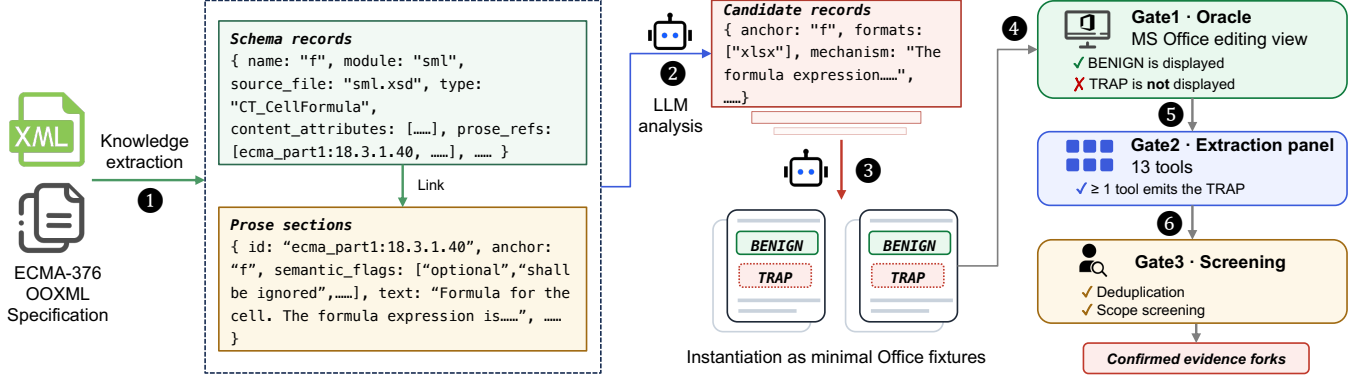


Figure 2: Overview of our workflow for discovering evidence forks from the OOXML specification.

format, the verbatim specification quote, the value Office renders, and the value an extractor reads. From these it writes a Python builder that produces the smallest Office file exercising that one construct. The builder uses format-specific Python libraries (python-docx, openpyxl, python-pptx) where the construct is exposed, and edits the underlying part XML in place where it is not (a webHidden run, an off-page framePr, an mc:AlternateContent branch). It places a unique BENIGN token in the Office-displayed view and a unique TRAP token in the alternative representation or channel. These distinct tokens let us test their presence or absence precisely in Office screenshots and extractor outputs. Within a three-attempt construction budget, 542 yield runnable fixtures; the remaining 97 either lack a suitable carrier for the controlled tokens or cannot be instantiated within that budget. Each successful run yields a candidate fixture for the two behavioral gates and Gate 3 screening.

Confirmation The confirmation stage consists of two behavioral gates followed by a deduplication and scope-screening.

Gate 1, the oracle, fixes what the default editing canvas shows: it must display the BENIGN token and not the TRAP. We automate this with a script that opens the file in Microsoft Office and captures its editing view. A multimodal LLM then reports whether the BENIGN and TRAP tokens are visible in the image. The judgment reduces to checking whether two distinct, attacker-chosen strings are present in a rendered screenshot rather than performing general visual recognition or reasoning. A candidate fixture passes Gate 1 only when BENIGN is visible and TRAP is not. After Gate 1, 412 candidate fixtures remain and proceed to Gate 2.

Gate 2, the extraction panel, requires that at least one extraction tool emit the TRAP. Table 1 lists the 13 tools, chosen to span what real ingestion stacks rely on: format-specific libraries, command-line converters, and the RAG loaders that front many LLM pipelines. One tool suffices: a single extractor that surfaces the trap makes that view available to the downstream model. We look up whether each trap token ap-

Table 1: The extraction tools panel.

Tool	Version	xlsx	docx	pptx
openpyxl [24]	3.1.5	✓		
python-calamine [44]	0.7.0	✓		
xlsx2csv [27]	0.8.6	✓		
python-docx [23]	1.2.0		✓	
docx2txt [26]	0.9		✓	
python-pptx [25]	1.0.2			✓
pandoc [28]	3.10		✓	
textutil	macOS 26.1		✓	
Apache Tika [29]	2.9.2	✓	✓	✓
LibreOffice [22]	26.2.4.2	✓	✓	✓
markdown [30]	0.1.6	✓	✓	✓
Unstructured [31]	0.23.1	✓	✓	✓
Docling [32]	2.108.0	✓	✓	✓

pears in a tool’s output after normalizing whitespace, rather than matching the whole string, so that harmless formatting differences in the BENIGN reading are not mistaken for a divergence. A candidate fixture passes Gate 2 when some tool emits the TRAP. After Gate 2, 163 candidate fixtures remain.

Gate 3 applies deduplication and scope screening to the candidates that pass Gates 1 and 2. We remove 104 duplicate records that exercise the same package-level construction. We retain candidates whose divergence follows from an OOXML-defined state, content role, or processing rule. We exclude purely visual camouflage, such as white-on-white text, object overlap, and extremely small text, because Office still renders the underlying content. We also exclude PowerPoint speaker notes, which Office presents in a dedicated notes area rather than as part of the slide, and reversible encodings that leave the trap visible. These scope criteria exclude 38 candidates, leaving 21 distinct mechanisms in the confirmed catalog.

The Confirmed Evidence Forks The confirmation workflow yields 21 evidence forks. Table 2 maps the identifiers used throughout the evaluation to short names, document

Table 2: Compact map of the mechanism identifiers used throughout the evaluation. Complete construct-level descriptions appear in Appendix A.

Class	ID	Fmt	Short name
Representation	M1	xlsx	Fixed-literal numFmt
	M2	xlsx	Blank numFmt
	M3	xlsx	Conditional-format override
	M4	xlsx	Formula / stale cache
	M5	docx	Binding / cached run
State	M6	xlsx	Hidden sheet
	M7	xlsx	Very-hidden sheet
	M8	xlsx	Filter-hidden rows
	M9	xlsx	Collapsed outline rows
	M10	xlsx	Hidden column
	M11	xlsx	Normal-view header
Visibility	M12	docx	Vanished run
	M13	pptx	Hidden shape
	M14	docx	Off-page text box
Compatibility	M15	docx	Unsatisfied MC Choice
	M16	docx	Unselected MC Fallback
Scope	M17	docx	Drawing alt text
	M18	docx	Table description
	M19	docx	Hyperlink display / target
Linearization	M20	xlsx	Covered merged-cell value
	M21	xlsx	Phonetic guide

formats, and view-construction classes. The detailed catalog in Appendix A identifies each OOXML construct in a specification-valid fixture and its empirically observed Office-versus-extractor divergence. We organize the mechanisms by the OOXML processing point at which the views diverge; Table 3 summarizes these six dimensions.

Representation (five) covers objects whose displayed or computed value is not the value stored: a custom number format rewrites the cell, an always-true conditional format overrides it, a formula recomputes over a stale cache, or a content control refreshes a data binding, while an extractor returns the raw stored value. *State* (six) hides content behind a container or view state that is present but not shown: a hidden or very-hidden sheet, an autofilter- or outline-collapsed row, a hidden column, a worksheet header absent from Normal view. *Visibility* (three) suppresses an object that would otherwise render, through a vanished run, a hidden shape, or a text box pushed off the page. *Compatibility* (two) covers Markup-Compatibility selection: Office retains the applicable branch, while an MC-blind extractor may emit content from an unselected branch. *Scope* (three) emits a non-body field as content: drawing alt text, a table description, a hyperlink target. *Linearization* (two) flattens structure, so that a value hidden under a merged cell, or a phonetic guide beside its base run, reaches the extractor out of place. State and representation are the largest classes, at six and five mechanisms; the rest hold two or three. By document type, twelve of the 21 occur in Excel, eight in Word, and one in PowerPoint.

Table 3: Six dimensions of OOXML view construction.

Class	View-construction step
Representation	Which encoded representation supplies an object’s value
State	Whether encoded content is active in the current application view
Visibility	Whether a realized object appears in the rendered view
Compatibility	How Markup Compatibility branch selection is applied
Scope	Which non-body fields count as document evidence
Linearization	How structured content is flattened into text

Representation mechanisms are particularly important because several do not rely on hidden or off-canvas content. A number-format or conditional-format override (M1, M3), a formula paired with a stale cache (M4), or a data binding (M5) places co-resident values or roles on one object. Office and an extractor can therefore expose different task evidence without suppressing or moving text. A search limited to hidden content would miss this structured part of the attack surface.

3.3 A Realistic Test Set

After confirming each fork on a minimal fixture, we embed it in realistic Office documents for evaluation. Content comes from TAT-QA [20], a JSON corpus of hybrid table-and-text excerpts from financial reports. For each sample, we render one excerpt as a new Excel, Word, or PowerPoint file in the mechanism’s target format and instantiate one evidence fork. This yields ten samples per mechanism, 210 in all. In each, the value in the Office-displayed view is the filing’s true value, while the extractor-emitted value or content is the attacker’s trap. Injections are minimal, so each file remains specification-valid and opens normally. Before its first web-chat upload, an author opens each instance in the default Microsoft Office editing view used by Gate 1 and manually verifies that the source content renders normally and that the planted content is not shown. All 210 instances pass this check. We separately validate all files with `OpenXmlValidator` from Microsoft Open XML SDK 3.5.1 [43], targeting Microsoft 365. It reports no package, schema, semantic, or Markup Compatibility errors for any file.

Trap placement follows two task families. In the *question-answering* family, the attacker uses the fork to substitute a single queried fact: a cell whose displayed value is the true figure but whose stored value reads a different number, or a counterpart whose visible name differs from the one an extractor emits. In the *summarization* family, the attacker uses the fork to plant a fabricated disclosure that is absent from the page yet present to an extractor, so that a model asked to summarize the filing reports a liability, provision, or related-party dealing the document never states. Three of these samples, one XLSX, one DOCX, and one PPTX, are walked through in Appendix D. Each sample carries a manifest with its task question, benign answer, and trap answer.

4 Measurement

We evaluate evidence-fork propagation, ecosystem reach, and detectability through five questions:

- RQ1.** Which extractors expose the trap-bearing view? (§4.2)
- RQ2.** Do end-to-end LLM services propagate that view into model outputs? (§4.3)
- RQ3.** Model or ingestion path: which sets exposure? (§4.4)
- RQ4.** How widely do affected ingestion paths appear in open-source agent and RAG projects? (§4.5)
- RQ5.** How common are these structures in ordinary files, and what do their benign base rates imply for presence-only screening? (§4.6)

4.1 Experimental Setup

Measurement Scope We evaluate the 21 evidence forks across extraction tools and LLM services on the 210 document instances of §3.3. The extraction panel comprises 13 tools (Table 1). We further test four native-ingestion APIs, meaning they accept and parse an uploaded Office document directly: GPT-5.5, Kimi K2.6, Qwen-doc-turbo, and GLM 5.2. Some mainstream APIs such as Anthropic’s Claude and Google’s Gemini do not accept Office documents for native parsing, so for those models we use their web chat interface: we test seven web chatbots (Claude Sonnet 5, ChatGPT GPT-5.5, DeepSeek V4, GLM 5.2, Gemini 3.5, Kimi K2.6, and Qwen-3.5) to assess each mechanism’s reachability and propagation in real deployments.

Trial Protocol For the API tests, we run each mechanism’s ten instances ten times, 2,100 runs per API and 8,400 trials in total. Across the seven web chatbots, we test each of the 210 documents once, yielding 1,470 web trials.

Response Labeling and Validation We use Claude Opus 4.8 and GPT-5.5 to independently label all 8,400 API responses under a binary rubric: TRAP means that a response asserts the planted content as a document fact; all other responses are NON-TRAP. Appendix C gives the complete prompt and input fields. The two judges agree on all 8,400 responses (100% raw agreement). We further audit five randomly sampled responses from each mechanism–API combination (420 total). The two authors split this audit set, and every human label agrees with both automated verdicts. The two authors manually label and verify every web response during testing; completing the 1,470 trials took about two weeks.

Controls We use two controls for the LLM measurement. First, we construct a matched benign twin for each of the 210 documents instantiating an evidence fork by removing the divergent representation or channel while preserving the source content and question, and query each API once. Second, as a visible-positive sanity check, we construct six non-divergent documents that place the designated target in ordinary vis-

ible cell, body, or slide text. Together, these controls span the three OOXML formats and both task families used in the main experiment.

4.2 RQ1: Extractor Reachability

RQ1 measures extractor reachability. Table 4 gives the full matrix; each tool runs at one pinned version with one set of call arguments, listed in Table 9 in Appendix B. Across the confirmed set, *width* (the number of applicable tools that emit the trap) ranges from a single tool to every applicable one. Each mechanism–tool outcome is identical across its ten base documents, indicating construct-stable behavior within our test set. The spread itself is unsurprising: read as a whole, though, the matrix reveals three structural properties that no single row or column shows.

The Format Decides Whether The Loader is a Security Choice

Read as a proportion of the tools that handle each format, reach is sharply bimodal (Figure 3): seven mechanisms fall between 11% and 33%, the other fourteen between 67% and 100%, and none lies in between. What separates the two bands is where the trap is carried: every mechanism in the low band sits outside the main content, in an XML attribute, a relationship target, an alternate-content branch, a worksheet header, or a phonetic annotation, while every mechanism in the high band is carried as ordinary content text that any walk of the document reaches. That is also what divides the two formats. A spreadsheet keeps almost all of its mechanisms in the cell grid, which retains visible and hidden cells in the same cell stream, so most extractors traverse both: the three native spreadsheet tools read seven mechanisms in common and differ on at most three, and five of the twelve spreadsheet mechanisms are read by every applicable tool. Word instead scatters its mechanisms across attributes, branches, and nested stories, and each library draws its own line on how much of the document tree to emit, so the four native Word extractors share exactly one mechanism, the vanished run, and agree on nothing else. The regularity holds inside a format as well as across formats, since the two spreadsheet mechanisms that live outside the grid fall to 25%, the same band as the Word attribute channels. The consequence is asymmetric: loader choice has little effect on high-reach XLSX mechanisms but substantially changes DOCX exposure.

Class and Reach Move Together Where a trap is carried is not an incidental property of a mechanism, it is what the classes of §3 are defined by. *Scope* means a non-body field is emitted as content, so its trap necessarily sits outside the main content; *compatibility* means a Markup-Compatibility branch is selected, so its trap necessarily sits in an alternate branch; *representation* means the displayed value differs from the stored one, so its trap is necessarily a stored value in the main content. The class fixes the carrier, and reach follows the carrier closely. Table 5 bears this out: ordering the six classes by mean reach reproduces exactly their ordering by

Table 4: Extractor reach. ● = the tool emits the trap; ○ = it handles the format but its output stays benign; blank = it does not handle the format. Denominators are format-aware: **Reach** counts tools per mechanism (8 XLSX, 9 DOCX, 6 PPTX), and the bottom row counts mechanisms per tool.

Mechanism	Format	openpyxl	calamine	xlsx2csv	python-docx	docx2txt	pandoc	textutil	python-pptx	markdown	Apache Tika	LibreOffice	Unstructured	Docling	Reach
M1	xlsx	●	●	●						●	●	●	●	●	8/8
M2	xlsx	●	●	●						●	●	●	●	●	8/8
M3	xlsx	●	●	●						●	●	●	●	●	8/8
M4	xlsx	●	●	●						●	●	●	●	●	8/8
M5	docx				○	●	●	●		●	●	○	○	●	6/9
M6	xlsx	●	●	●						●	●	○	●	○	6/8
M7	xlsx	●	●	●						●	●	○	●	○	6/8
M8	xlsx	●	●	○						●	●	●	●	●	7/8
M9	xlsx	●	●	○						●	●	●	●	●	7/8
M10	xlsx	●	●	●						●	●	●	●	●	8/8
M11	xlsx	●	○	○						○	●	○	○	○	2/8
M12	docx				●	●	●	●	●	●	●	●	●	●	9/9
M13	pptx								●	●	●	○	●	●	5/6
M14	docx				○	●	●	●		●	●	○	○	○	6/9
M15	docx				○	●	○	●		○	○	○	○	○	2/9
M16	docx				○	●	○	●		●	○	○	○	○	3/9
M17	docx				○	○	●	○		●	○	○	○	○	2/9
M18	docx				○	○	●	○		○	○	○	○	○	1/9
M19	docx				○	○	○	○		●	○	○	○	●	2/9
M20	xlsx	○	●	●						●	●	●	●	○	6/8
M21	xlsx	○	○	●						○	●	○	○	○	2/8
Exposed		10/12	10/12	9/12	1/8	5/8	5/8	5/8	1/1	17/21	16/21	9/21	12/21	12/21	

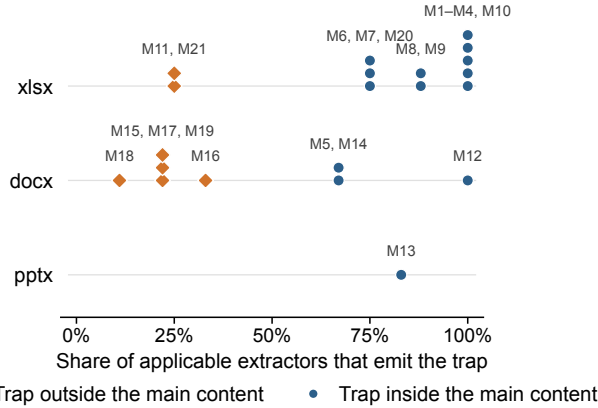


Figure 3: Reach as a proportion of the tools that handle each format; each marker is one mechanism.

how much of each class sits inside the main content, from representation at 0.93 with all five mechanisms inside, down to scope at 0.19 with none. The two orderings coincide across all six classes, so the classes group mechanisms that behave alike at the extraction layer rather than merely sorting them for presentation.

Table 5: Mean extractor reach by class, against the share of each class whose trap is carried inside the main content.

Class	Mech.	Mean reach	Inside main content
Representation	5	0.93	100%
Visibility	3	0.83	100%
State	6	0.75	83%
Linearization	2	0.50	50%
Compatibility	2	0.28	0%
Scope	3	0.19	0%

Broader Coverage Expands Exposure The one Word mechanism read by every applicable extractor is M12, the vanished run. The other Word mechanisms are visible to some extractors and invisible to others. `python-docx` reads only M12 because it does not surface bindings, alternate branches, or accessibility attributes. General-purpose converters designed for broader content coverage expose more mechanisms. The two broadest panel members, `markdown` and `Apache Tika`, expose 17 and 16 mechanisms. Their flattened outputs surface more OOXML channels without preserving their Office-assigned roles. Text coverage and semantic integrity therefore pull against each other, and a pipeline that upgrades its loader for better coverage can silently widen its attack surface.

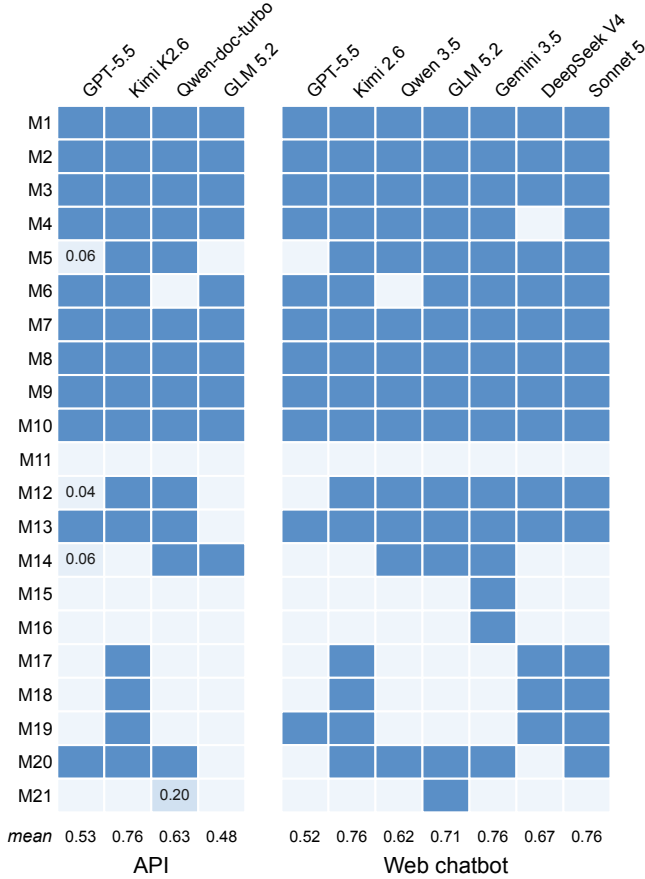


Figure 4: Per-mechanism propagation into LLM output. Each cell is the fraction of trials in which the service reports the trap as a document fact.

Finding I. Broader text traversal does not imply more faithful view construction: broad converters expose more trap-bearing channels, while narrow loaders can appear safer simply by omitting document semantics. Extractor consensus therefore reflects shared traversal behavior, not agreement with the Office view.

4.3 RQ2: Propagation into LLM Outputs

RQ2 asks whether a trap an extractor emits is actually adopted by a model as a document fact. We put all 21 mechanisms through eleven interfaces: four native-ingestion APIs and seven web chatbots. Figure 4 gives every cell. On our mechanism-balanced set, mean record-level propagation ranges from 0.48 (GLM) to 0.76 (Kimi) across the APIs and from 0.52 to 0.76 across the web chatbots. Each mechanism contributes ten records equally, with API outcomes averaged over ten repetitions, so these rates characterize our evaluation set rather than real-world attack prevalence. Across the 840

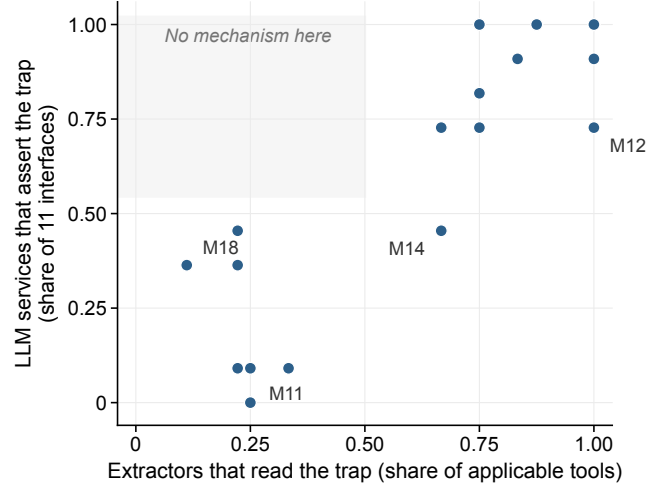


Figure 5: Extractor reach and LLM-interface exposure. Each point is one mechanism: how many applicable panel tools emit its trap, against how many of the eleven LLM interfaces assert that trap as a document fact.

matched-clean document-API trials, no response surfaces the absent trap (0/840). Conversely, all four APIs surface the designated target in every visible-positive trial (24/24 document-API trials). These controls bound the main condition: the targets are not produced on their matched clean twins, yet are recoverable through all four APIs when present in ordinary visible content.

Extractor Reach Tracks LLM-Interface Exposure Figure 5 shows a strong observed association between the two layers. Of the fourteen mechanisms that most applicable panel tools read, every one is asserted by between five and eleven of the eleven LLM interfaces. Of the seven that few panel tools read, none reaches more than five interfaces. Mechanisms with broader panel reach thus tend to appear across more LLM interfaces. This relationship is observational: an LLM service may use an extractor or configuration outside our panel, and broad panel reach does not guarantee broad propagation. The table description (M18), for example, is read by exactly one panel tool but propagates on four interfaces. Overall, twenty of the twenty-one mechanisms are asserted as fact by at least one interface; the exception is the worksheet header (M11), which offline tools read but no tested service surfaces as model evidence.

Propagation Is Stable Across Tested Conditions The API and web experiments provide two distinct forms of stability evidence, summarized in Figure 6. For the APIs, each document is queried ten times. Of the 840 document-API pairs, 826 (98.3%) produce the same binary verdict in all ten trials; 502 always surface the trap, 324 never do, and 14 vary across repetitions. For the web chatbots, each mechanism-interface cell contains ten distinct documents tested once each. All 147

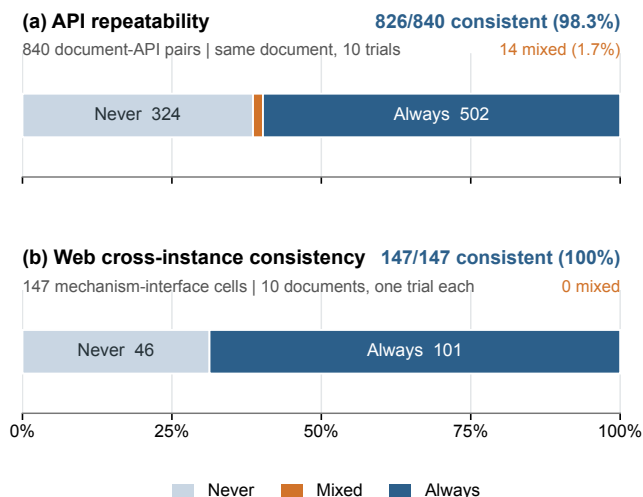


Figure 6: Stability under two evaluation designs. API units repeat the same document ten times; web units contain ten distinct documents tested once each. *Never* and *Always* are consistent outcomes; *Mixed* contains both verdicts.

cells are consistent across those instances: 101 surface the trap for all ten documents and 46 for none. Thus, the API results measure within-document repeatability, whereas the web results measure consistency across document instances.

Aggregate Web Coverage Is Broader Across the two panels in Figure 4, the seven web interfaces collectively expose 20 of the 21 mechanisms, while the four APIs collectively expose 18. The difference is M15 and M16, the two Markup-Compatibility branches, which appear only in the web panel. This containment holds only for the aggregate unions: it does not imply that a vendor’s API is a lower bound on its web exposure. As RQ3 shows, GPT’s two entry paths differ in both directions, while Qwen’s API exposes M21 and its web interface does not.

Finding II. Broad upstream reach tends to become broad downstream exposure: LLM interfaces generally preserve, rather than reconcile, the evidence selected during ingestion. Persistence across API retries and distinct web submissions points to the pipeline, not generation noise.

4.4 RQ3: Model or Ingestion Layer?

An LLM interface couples a model with an ingestion path. We ask which of the two more closely tracks the observed exposure pattern.

Same Vendor, Different Entry Points Figure 4 shows that one service can behave differently at its two entry points. GLM’s web interface and file-parsing API both advertise GLM 5.2, yet the web interface exposes five mechanisms

(M5, M12, M13, M20, M21) that the API does not, while the API exposes nothing that the web does not. Holding the advertised model label constant localizes this difference to the end-to-end entry path, although it does not identify which backend component differs. GPT-5.5 also differs across its two entry points, though by less: its web interface exposes eleven mechanisms and its API fourteen. What stands out on the API is three mechanisms (M5, M12, M14), that it exposes only rarely, on four to six of a hundred runs rather than all or none. We re-ran these three and asked the model to return the document text verbatim. The trap value appeared in the returned text in only a small fraction of runs while the surrounding text stayed the same across runs. This pattern implicates upstream file handling rather than ordinary answer stochasticity, although it does not identify the responsible backend component.

Not every service splits this way. Kimi K2.6 exposes the same sixteen mechanisms through its web interface and its API, a result consistent with a shared or behaviorally equivalent ingestion path. Qwen differs at a single mechanism (M21), exposed in 20% of API trials but not through the web interface; this comparison is weaker than the others, because Qwen’s native file-parsing API is Qwen-doc-turbo while its web chat offers no Qwen-doc-turbo, so the web tests use Qwen-3.5 and the two ends differ in model as well as path. Taken together, the same vendor’s API and web chat can read the same file differently.

Exposure is far steadier across versions of one model than across entry points. Holding the OpenAI file-input path fixed, we ran the full corpus of 210 documents ten times each on gpt-5.4 and again on gpt-5.5, 2,100 judged runs per model. Not one mechanism flips. A version apart on the same path, the extraction behaves the same and so does the attack surface. Across these controls, exposure tracks the tested ingestion path more closely than model version alone. The clearest case is one where that path is published, which we turn to next.

Three Claude Models, One Skill The strongest form of this control is inside a single vendor. Anthropic serves Office documents to Claude application through a document skill, never through its API, which rejects OOXML at validation; the skill reads XLSX with `openpyxl`, DOCX with `pandoc`, and PPTX with `markitdown` [45], so the text a model sees is fixed before it reasons. We ran three Claude models spanning that vendor’s range, Haiku 4.5, Sonnet 5, and Opus 4.8, through this shared path, and all three expose the same sixteen of the twenty-one mechanisms, mechanism for mechanism (Table 6). Under this shared path, model choice does not change the exposed set.

Version and Invocation Change What a Tool Reads Claude reads DOCX through `pandoc`, and so does our panel, yet on one mechanism the two disagree. Our `pandoc` reads the off-page text box (M14) on every sample (Table 4), while Claude Sonnet 5 never exposes it (Figure 4). Claude’s published skill confirms the library, but its behavior differs from our pinned version. Reading M14 changed between the two

Table 6: Three Claude models of different capability, one ingestion path. All three read Office through the same claude.ai document skill (openpyxl/pandoc/markitdown).

Model	Mechanisms exposed
Haiku 4.5	16/21
Sonnet 5	16/21
Opus 4.8	16/21
Same exposed set	M1–M10, M12–M13, M17–M20

pandoc releases we ran, 3.8.3 missing it on every sample and 3.10 reading it on every one, so our 3.10 panel surfaces M14 while Claude’s pattern is consistent with an older build. Two more cases have the same shape. A hyperlink target (M19) is dropped by `pandoc -t plain` but emitted by `-t markdown`, the format an LLM pipeline actually wants. A merged-cell value (M20) is masked by `openpyxl`’s default load yet surfaced in its streaming `read_only` mode, the one used to dump large sheets. Version, output format, and read mode change which document evidence reaches the model, so knowing that a service uses `pandoc` or `openpyxl` says little until we also know the version it pins and the arguments it passes.

Behavior-Level Matching of Closed Ingestion Paths A mechanism can be read differently not only by different extraction tools but by different versions and arguments of one tool, so the set of mechanisms a service surfaces carries information about how it reads documents. We therefore ask whether that pattern can match a closed service to tested candidate configurations, including versions and invocation modes. The pattern is usable because each tool’s per-mechanism outcome is deterministic, making it a stable behavioral signature over our probes. Claude’s published skill, for instance, names three libraries but not the version or read mode each runs at, and a fully closed service names none, so there is useful behavior to compare. We run this test on DOCX, because that is where the extractors diverge. In Table 4 the eight spreadsheet tools collapse to six signatures, three of them identical, so an XLSX pattern separates almost nothing. The Word tools instead each read a different subset: of the eight DOCX mechanisms, two are read by nearly every tool and tell us nothing, while the other six split the tools apart, and one of those, the table description (M18), is read by `pandoc` alone. A DOCX pattern is therefore a useful behavioral signature (Figure 7).

Matched against these signatures, 8 of the 11 interfaces match at least one tested candidate configuration over the eight probes. Four share a vector with `pandoc`: Kimi K2.6 (API), Sonnet 5 (web), DeepSeek V4 (web), and Kimi K2.6 (web). Gemini 3.5 (web) matches `docx2txt` or `textutil`; Qwen-doc (API), GLM 5.2 (web), and Qwen 3.5 (web) match Apache Tika. The four `pandoc`-like interfaces can be matched to a more specific tested configuration: their vector differs from our pinned `pandoc` at two mechanisms, while `pandoc`

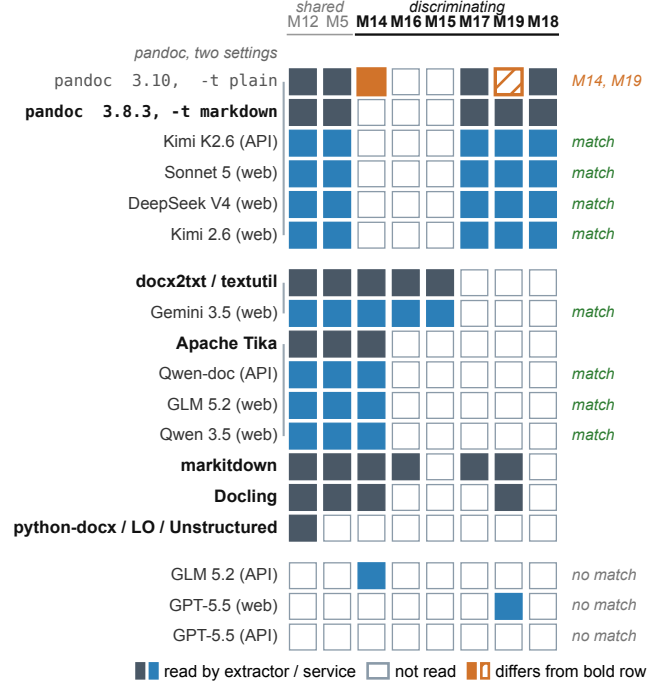


Figure 7: Behavior-level matching between LLM interfaces and candidate DOCX extraction configurations. Dark rows denote tested configurations; blue rows share their eight-probe vectors. A match does not identify the backend.

3.8.3 with Markdown output reproduces it on all eight. Markdown carries the hyperlink target (M19), and only the newer version reads the off-page text box (M14). This establishes behavioral equivalence over our probes, not backend identity.

Claude provides the only external check: its published skill names `pandoc`, and its observed vector matches a tested `pandoc` configuration. This validates the library-level match but not the exact version or invocation, so we claim an existence proof of behavior-level candidate matching rather than a fingerprinting benchmark. The more mechanisms a panel distinguishes, the more tightly it can narrow the configurations consistent with a service’s behavior. An attacker who cannot see a service’s stack can use a fixed probe set to narrow those candidates and prioritize attacks known to affect them without learning the backend’s identity.

Finding III. Model identity and extractor name are both too coarse: evidence-fork exposure depends on the full ingestion configuration, including version, output mode, and arguments. The resulting stable exposure vector lets us use evidence forks to narrow a closed service to behaviorally equivalent candidate configurations without identifying its backend.

4.5 RQ4: How Widely Do Affected Ingestion Paths Appear in Open-Source Projects?

We answer this with a source-level survey, not deployment telemetry or end-to-end measurements. At one pinned commit per repository (July 2026), we inventory the default OOXML ingestion paths of sixteen popular open-source agent and RAG projects and follow each import chain to the *leaf* library that turns OOXML into text. These mappings identify reachable extractor families, not propagation through the complete application (Table 7). We stop at a library only when it does the extraction itself. markdown, Unstructured, and Apache Tika are recorded as they are called, because each rewrites the document into its own normalized text with its own reach. A thin pass-through is instead resolved to what it actually loads: the six applications that read spreadsheets through pandas (LangChain, LlamaIndex, Haystack, Dify, Kotaemon, Verba) are listed as openpyxl, the engine pandas loads by default, since none selects the alternative calamine engine and pandas contributes no extraction of its own.

The sixteen projects concentrate on a few leaf extractor families. Their default paths therefore map to families with the reach profiles measured in RQ1, although application versions, invocation arguments, and downstream processing may change the resulting exposure. markdown has the widest panel reach, 17 of the 21 mechanisms, and four projects route through it, including AutoGen, GraphRAG, and PrivateGPT. Separately, txtai is the only surveyed project whose default path reaches Apache Tika (16/21), the only panel tool that surfaces the phonetic guide and worksheet header.

The source mappings preserve RQ1’s format asymmetry: Word paths span families with widely different measured reach, whereas the spreadsheet families all have broad panel reach. Switching among the surveyed spreadsheet defaults is therefore unlikely to eliminate potential exposure, although exact application behavior still depends on versions, arguments, and downstream processing.

Finding IV. Project diversity masks dependency concentration: default OOXML paths converge on a small set of shared extractor families. This concentration has format-dependent consequences: changing the Word loader can substantially alter potential exposure, whereas the surveyed spreadsheet paths all resolve to families with broad panel reach.

4.6 RQ5: How Common Are These Mechanisms in Ordinary Documents?

A defense that only strips invisible text misses most of these mechanisms, because most leave both the shown value and the extracted value in the file, while the ingestion path exposes only one to the model. What matters is not whether text is hidden but how often each construction appears in ordinary

Table 7: The leaf extractor(s) reached by each surveyed project’s default OOXML ingestion path, as resolved from source code. GitHub stars were measured on July 26, 2026.

Application	Leaf extractor(s)	Stars
AutoGen [46]	markdown	60.0K
GraphRAG [47]	markdown	34.9K
PrivateGPT [48]	markdown	57.4K
Onyx [49]	markdown, openpyxl	31.2K
LangChain [3]	docx2txt, openpyxl, python-pptx	142.6K
LlamaIndex [4]	docx2txt, openpyxl, python-pptx	51.1K
Quivr [50]	docx2txt, Unstructured	39.4K
Embedchain [51]	docx2txt, Unstructured	61.8K
Haystack [52]	python-docx, openpyxl, python-pptx	26.0K
RAGFlow [53]	python-docx, openpyxl, python-pptx	86.1K
R2R [54]	python-docx, openpyxl, python-pptx	7.9K
Dify [55]	python-docx, openpyxl, Unstructured	150.3K
Verba [56]	python-docx, openpyxl	7.7K
CrewAI [57]	python-docx	56.2K
Kotaemon [58]	Unstructured, openpyxl	25.6K
txtai [59]	Apache Tika	12.8K

documents: rare constructions can support selective triage, while common ones cannot be distinguished by presence alone. RQ5 measures this base rate. We write one static rule per mechanism that flags its construction, and run the probe over fixed-seed random samples of 2,000 XLSX files from Sheetpedia [60] and 2,000 DOCX files from DocxCorp [61]. Both are public datasets with stable dataset records and downloadable manifests. For PowerPoint, we retain a separate set of 263 public PPTX files collected from national-government portals and research repositories [62–67]. This smaller PPTX set is heterogeneous and serves only as the denominator for M13. We validate every retained file as an OOXML package and remove exact duplicates by SHA-256. Each rule fires on all constructed evidence-fork instances, so it matches the construction faithfully; on ordinary files the same rule measures how often that construction occurs on its own, not whether it is malicious. Every construction has legitimate uses, a hidden sheet holding a lookup table, an image carrying alt-text, a merged title cell, so the result is a base rate.

The base rates are sharply bimodal (Figure 8). Six mechanisms were not observed in any file in their corresponding format samples, and ten more appeared in under 2%. The remaining five range from 2.75% to 16.5%: worksheet headers, hidden columns, compatibility fallbacks, labeled hyperlinks, and image alt-text. Across the three format-specific samples, 748 of 4,263 documents (17.5%) carry at least one signature. Because the format sample sizes are chosen independently, this aggregate is descriptive rather than a population estimate.

The split says what a presence scan can and cannot do. The sixteen rare signatures provide selective triage signals in these samples; this covers every number-format override, stale cache, data binding, merged-cell value, and hidden sheet,

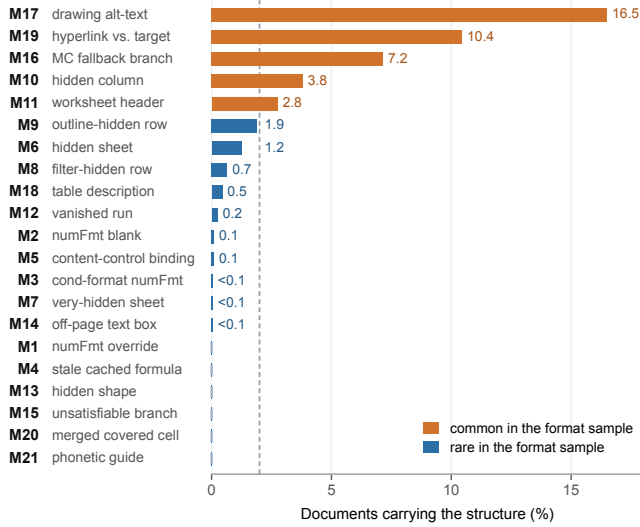


Figure 8: Share of documents carrying each mechanism’s structural signature, measured against its own format sample: 2,000 Sheetpedia XLSX, 2,000 DocxCorp DOCX, or 263 public PPTX files.

row, or shape. The five common signatures mix metadata and non-body channels with routine spreadsheet state, and occur in 2.75% to 16.5% of their format samples. A presence flag is therefore not actionable for them. For these common constructions, malicious and benign documents can use the same structural feature; their difference lies in the content and its intended role, which presence alone does not capture. Telling attack from benign use there needs the content itself, or provenance that marks text drawn from a channel the reader never sees so a downstream model can discount it rather than accept it as body content.

Finding V. Benign base rates split the mechanisms into two regimes for presence-only screening: structural presence is a usable triage signal for the rare constructions but cannot separate a planted fork from ordinary use on the common channels. Discriminating the latter requires content and role, not presence alone.

5 Discussion and Limitations

View Construction Is a Security Policy An OOXML loader is usually treated as a preprocessing utility, but it decides which document view becomes model evidence. RQ3 shows that this decision belongs to the full ingestion configuration: entry point, extractor version, output mode, and arguments can all change exposure, while models sharing one path can retain the same exposed set. RQ4 shows why this matters beyond one service: otherwise diverse projects repeatedly route

OOXML through a small set of extractor families. Loader selection and upgrades should therefore be treated as security-relevant policy changes rather than interchangeable implementation details. The stable vectors observed under API repetition and across web instances also suggest a practical audit primitive: a fixed evidence-fork suite can regression-test a pipeline’s evidence-selection behavior and detect drift without backend access.

Coverage Is Not Semantic Fidelity RQ1 shows that recovering more strings can widen the set of semantic roles flattened into model evidence. Narrow extraction is not a general remedy: omitting document semantics does not restore fidelity to the Office view. Text coverage and semantic fidelity are therefore distinct objectives. RQ5 shows why defense cannot stop at structural presence: rare constructions can support selective alerts, while common channels also serve routine document functions. A promising direction for future work is a two-part defense: cheap presence checks for rare constructions, paired with role-aware consistency checks for common ones. Such a scheme would likely need emitted evidence to retain its source and semantic role, and the ingestion contract to make representation selection explicit. We leave building and evaluating it to future work.

Limitations Because each mechanism contributes equally, aggregate propagation rates characterize our mechanism-balanced evaluation set rather than the distribution of documents in deployment. We therefore interpret them together with per-mechanism outcomes and interface coverage, rather than as population-level risk estimates. Results for tools, services, and project dependencies reflect the pinned or dated configurations evaluated in this study. Implementations may evolve and expose different subsets, so product-level mappings are snapshots rather than permanent rankings. The evidence-fork catalog and analysis method do not depend on any one vendor configuration.

6 Conclusion

OOXML-to-LLM pipelines can construct model evidence that diverges from the Office view presented for review. We systematized this failure as evidence forks and confirmed 21 specification-grounded mechanisms across Excel, Word, and PowerPoint. Cross-stack measurements show that exposure follows the ingestion configuration, not model identity alone, and that shared extractor families recur across open-source projects. Secure ingestion therefore needs an explicit contract that defines the intended view, preserves evidence provenance and semantic roles, and surfaces rather than silently flattens disagreements. Evidence-fork probes provide a practical way to test that contract as loaders evolve.

References

- [1] Statista. Number of Office 365 users worldwide, by country. <https://www.statista.com/statistics/983321/worldwide-office-365-user-numbers-by-country/>, 2025. Accessed July 2026.
- [2] Humza Naveed, Asad Ullah Khan, Shi Qiu, Muhammad Saqib, Saeed Anwar, Muhammad Usman, Naveed Akhtar, Nick Barnes, and Ajmal Mian. A Comprehensive Overview of Large Language Models. *ACM Transactions on Intelligent Systems and Technology*, 16(5):1–72, 2025.
- [3] LangChain. LangChain: Building applications with LLMs. <https://github.com/langchain-ai/langchain>, 2025.
- [4] LlamaIndex. LlamaIndex: Data framework for LLM applications. https://github.com/run-llama/llama_index, 2025.
- [5] Haoyu Dong, Jianbo Zhao, Yuzhang Tian, Junyu Xiong, Mengyu Zhou, Yun Lin, José Cambronero, Yeye He, Shi Han, and Dongmei Zhang. Encoding Spreadsheets for Large Language Models. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP 2024)*, 2024.
- [6] Yifei Dong, Fengyi Wu, Kunlin Zhang, Yilong Dai, Sanjian Zhang, Wanghao Ye, Sihan Chen, and Zhi-Qi Cheng. Large Language Model Agents in Finance: A Survey Bridging Research, Practice, and Real-World Deployment. In *Findings of the Association for Computational Linguistics: EMNLP 2025*, 2025.
- [7] Ian Markwood, Dakun Shen, Yao Liu, and Zhuo Lu. PDF Mirage: Content Masking Attack Against Information-Based Online Services. In *Proceedings of the 26th USENIX Security Symposium (SENIX Security '17)*, pages 833–847, 2017.
- [8] Side Liu and Jiang Ming. Semantic Integrity Failures in Document-to-LLM Supply Chains. arXiv preprint arXiv:2606.15020, 2026.
- [9] Hyundong Jin, Sicheol Sung, Shinwoo Park, SeungYeop Baik, and Yo-Sub Han. TRAPDOC: Deceiving LLM Users by Injecting Imperceptible Phantom Tokens into Documents. In *Findings of the Association for Computational Linguistics: EMNLP 2025*, 2025.
- [10] Junjie Xiong, Changjia Zhu, Shuhang Lin, Chong Zhang, Yongfeng Zhang, Yao Liu, and Lingyao Li. Invisible Prompts, Visible Threats: Malicious Font Injection in External Resources for Large Language Models. In *Findings of the Association for Computational Linguistics: EMNLP 2025*, 2025.
- [11] ECMA International. ECMA-376: Office Open XML File Formats. <https://www.ecma-international.org/publications-and-standards/standards/ecma-376/>, 2016.
- [12] ISO/IEC. ISO/IEC 29500: Information technology — Document description and processing languages — Office Open XML File Formats. <https://www.iso.org/standard/71691.html>, 2016.
- [13] Jens Müller, Fabian Ising, Christian Mainka, Vladislav Mladenov, Sebastian Schinzel, and Jörg Schwenk. Office Document Security and Privacy. In *Proceedings of the 14th USENIX Workshop on Offensive Technologies (WOOT '20)*, 2020.
- [14] Simon Rohlmann, Vladislav Mladenov, Christian Mainka, Daniel Hirschberger, and Jörg Schwenk. Every Signature is Broken: On the Insecurity of Microsoft Office’s OOXML Signatures. In *Proceedings of the 32nd USENIX Security Symposium (USENIX Security '23)*, 2023.
- [15] Yufan You, Jianjun Chen, Qi Wang, and Haixin Duan. My ZIP Isn’t Your ZIP: Identifying and Exploiting Semantic Gaps Between ZIP Parsers. In *Proceedings of the 34th USENIX Security Symposium (USENIX Security '25)*, 2025.
- [16] Aniello Castiglione, Bonaventura D’Alessio, Alfredo De Santis, and Francesco Palmieri. New Steganographic Techniques for the OOXML File Format. In *Availability, Reliability and Security for Business, Enterprise and Health Information Systems (CD-ARES)*, 2011.
- [17] Alberto Castagnaro, Umberto Salviati, Mauro Conti, Luca Pajola, and Simeone Pizzi. The Hidden Threat in Plain Text: Attacking RAG Data Loaders. In *Proceedings of the 18th ACM Workshop on Artificial Intelligence and Security (AISec '25)*, 2025.
- [18] Dorina Sili, Bastian Küppers, and Theodor Schnitzler. Universal Prompt Injection Techniques for Detecting LLM-Based Assignment Fraud. In *Benelux Conference on Artificial Intelligence (BNAIC)*, 2025.
- [19] Bruno Martins Tomazini, Larissa Bianchini, Lucas Traujas, Natalia Vieira Rodrigues, Felipe Guindalini Lorenzato, Mariana Silveira de Alcantara Chaud, Karoline Cordeiro Vercka Gervásio, Israel Silva Maia, Fernando José da Silva Ramos, Bruno Adler Maccagnan Pinheiro Besen, et al. Susceptibility of Large Language Models to Hidden Nudge Injection During Simulated Medical Peer Review: A Quasi-Experimental Study. *Research Integrity and Peer Review*, 2026.

- [20] Fengbin Zhu, Wenqiang Lei, Youcheng Huang, Chao Wang, Shuo Zhang, Jiancheng Lv, Fuli Feng, and Tat-Seng Chua. TAT-QA: A question answering benchmark on a hybrid of tabular and textual content in finance. In *Proc. 59th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2021.
- [21] Microsoft. Microsoft Office (Microsoft 365). <https://www.microsoft.com/microsoft-365>, 2025.
- [22] The Document Foundation. LibreOffice: The free and open source office suite. <https://www.libreoffice.org/>, 2025.
- [23] python-docx. python-docx: Create and update Microsoft Word .docx files. <https://github.com/python-openxml/python-docx>, 2025.
- [24] openpyxl. openpyxl: A Python library to read/write Excel 2010 xlsx/xlsm files. <https://openpyxl.readthedocs.io/>, 2025.
- [25] python-pptx. python-pptx: Create and update PowerPoint .pptx files. <https://python-pptx.readthedocs.io/>, 2025.
- [26] docx2txt. docx2txt: A pure Python utility to extract text from .docx files. <https://github.com/ankushshah89/python-docx2txt>, 2025.
- [27] xlsx2csv. xlsx2csv: A fast xlsx to csv converter. <https://github.com/dilshod/xlsx2csv>, 2025.
- [28] John MacFarlane. Pandoc: A universal document converter. <https://pandoc.org/>, 2025.
- [29] Apache Software Foundation. Apache Tika: A content analysis toolkit. <https://tika.apache.org/>, 2025.
- [30] Microsoft. MarkItDown: Convert files to Markdown for LLMs. <https://github.com/microsoft/markitdown>, 2025.
- [31] Unstructured.io. Unstructured: Open-source libraries for ingesting and pre-processing documents. <https://github.com/Unstructured-io/unstructured>, 2025.
- [32] IBM. Docling: Document parsing for generative AI. <https://github.com/docling-project/docling>, 2025.
- [33] Nicola Ruaro, Fabio Pagani, Stefano Ortolani, Christopher Kruegel, and Giovanni Vigna. SYMBEXCEL: Automated Analysis and Understanding of Malicious Excel 4.0 Macros. In *Proceedings of the 43rd IEEE Symposium on Security and Privacy (S&P '22)*, 2022.
- [34] Zhuotao Lian, Weiyu Wang, Qingkui Zeng, Toru Nakanishi, Teruaki Kitasuka, and Chunhua Su. Prompt-in-Content Attacks: Exploiting Uploaded Inputs to Hijack LLM Behavior. In *Proceedings of the International Conference on Network and System Security (NSS '25)*, 2025.
- [35] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. Not What You've Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec '23)*, 2023.
- [36] Yang Feng and Xudong Pan. StruPhantom: Evolutionary Injection Attacks on Black-Box Tabular Agents Powered by Large Language Models. arXiv preprint arXiv:2504.09841, 2025.
- [37] Jonas Heß and Kalman Graffi. Detection of Malicious Office Open Documents (OOXML) Using Large Language Models: A Static Analysis Approach. *Journal of Cybersecurity and Privacy*, 5(2):32, 2025.
- [38] Toby Murray. PhantomLint: Principled Detection of Hidden LLM Prompts in Structured Documents. arXiv preprint arXiv:2508.17884, 2025.
- [39] Mohan Zhang, Yuqi Jia, Zhen Tan, Steven Jiang, Neil Zhenqiang Gong, Tianlong Chen, and Dawn Song. Measuring Real-World Prompt Injection Attacks in LLM-based Resume Screening. In *Proceedings of the 35th USENIX Security Symposium (USENIX Security '26)*, 2026.
- [40] Microsoft. [MS-DOCX]: Word Extensions to the Office Open XML (.docx) File Format. https://learn.microsoft.com/en-us/openspecs/office_standards/ms-docx/, 2025.
- [41] Microsoft. [MS-XLSX]: Excel (.xlsx) Extensions to the Office Open XML SpreadsheetML File Format. https://learn.microsoft.com/en-us/openspecs/office_standards/ms-xlsx/, 2025.
- [42] Microsoft. [MS-PPTX]: PowerPoint (.pptx) Extensions to the Office Open XML File Format. https://learn.microsoft.com/en-us/openspecs/office_standards/ms-pptx/efd8bb2d-d888-4e2e-af25-cad476730c9f, 2025.
- [43] Microsoft. Open XML SDK. <https://github.com/dotnet/Open-XML-SDK>, 2024.
- [44] python-calamine. python-calamine: Python bindings for the calamine spreadsheet reader. <https://github.com/dimastbk/python-calamine>, 2025.

- [45] Anthropic. Agent Skills: Docx, Xlsx, and Pptx Document Skills. <https://github.com/anthropics/skills>, 2026.
- [46] Microsoft. AutoGen: A programming framework for agentic AI. <https://github.com/microsoft/autogen>, 2025.
- [47] Microsoft. GraphRAG: A modular graph-based RAG system. <https://github.com/microsoft/graphrag>, 2025.
- [48] Zylon. PrivateGPT: Document Q&A with private LLMs. <https://github.com/zylon-ai/private-gpt>, 2025.
- [49] Onyx. Onyx (formerly Danswer): Gen-AI and enterprise search. <https://github.com/onyx-dot-app/onyx>, 2025.
- [50] Quivr. Quivr: Opinionated RAG for integrating GenAI in your apps. <https://github.com/QuivrHQ/quivr>, 2025.
- [51] Embedchain. Embedchain: A framework to create RAG applications. <https://github.com/embedchain/embedchain>, 2025.
- [52] deepset. Haystack: LLM orchestration framework for RAG and agents. <https://github.com/deepset-ai/haystack>, 2025.
- [53] InfiniFlow. RAGFlow: An open-source RAG engine based on deep document understanding. <https://github.com/infiniflow/ragflow>, 2025.
- [54] SciPhi AI. R2R: The most advanced AI retrieval system. <https://github.com/SciPhi-AI/R2R>, 2025.
- [55] LangGenius. Dify: An open-source LLM app development platform. <https://github.com/langgenius/dify>, 2025.
- [56] Weaviate. Verba: The golden RAGriever. <https://github.com/weaviate/Verba>, 2025.
- [57] crewAI. CrewAI: Framework for orchestrating role-playing autonomous AI agents. <https://github.com/crewAIInc/crewAI>, 2025.
- [58] Cinnamon. Kotaemon: An open-source RAG UI for chatting with documents. <https://github.com/Cinnamon/kotaemon>, 2025.
- [59] NeuML. txtai: All-in-one embeddings database. <https://github.com/neuml/txtai>, 2025.
- [60] Zailong Tian, Zhuoheng Han, Houfeng Wang, and Lizi Liao. Sheeetpedia: A 300K-Spreadsheet Corpus for Spreadsheet Intelligence and LLM Fine-Tuning. In *Advances in Neural Information Processing Systems*, 2025.
- [61] SuperDoc. docx-corpus: A Classified Corpus of Word Documents. <https://huggingface.co/datasets/superdoc-dev/docx-corpus>, 2026.
- [62] UK Government. data.gov.uk: Find open data. <https://www.data.gov.uk>, 2025.
- [63] Government of Canada. Open Government Portal. <https://open.canada.ca/data>, 2025.
- [64] Australian Government. data.gov.au: Australian Government open data. <https://data.gov.au>, 2025.
- [65] Harvard Dataverse. Harvard Dataverse Repository. <https://dataverse.harvard.edu/>, 2026.
- [66] CERN. Zenodo Research Repository. <https://zenodo.org/>, 2026.
- [67] Digital Science. Figshare Research Repository. <https://figshare.com/>, 2026.

Ethical Considerations

Stakeholders and Impacts The stakeholders are users and organizations that rely on OOXML-to-LLM workflows; LLM providers and extractor maintainers; other users of the measured services; the research team; and the broader security community. The principal benefit is to expose an ingestion risk that is otherwise invisible to document reviewers and to support defensive testing. Potential harms include the provider cost of live measurements, the risk of over-attributing behavior to a particular service, and dual-use knowledge that could help create misleading documents. We considered beneficence, respect for persons, and respect for law and the public interest in designing the study and the mitigations below.

Live-System Testing Offline extractors cannot establish how proprietary end-to-end ingestion paths handle an uploaded document, so the RQ2 and RQ3 measurements required limited testing of live services. We used the authors’ own accounts and only the API and web-upload functionality available to those accounts. All documents contained controlled, non-harmful factual substitutions or summary statements, and all prompts requested ordinary factual question answering or summarization. We included no attack instructions, prompt injections, jailbreaks, malware, macros, or requests for harmful content. We did not attempt penetration, account compromise, bypass of access controls, data exfiltration, service disruption, or access to other users’ or confidential data. Testing was limited to the pre-specified trial matrix in §4.1; we did

not stress-test the services or intentionally alter shared state. The remaining provider impact was the ordinary computation required to process our uploads and queries.

Human Subjects and Data We recruited no external participants and collected no data from service users; human review was performed only by the authors on Office renderings and model responses. The realistic fixtures use excerpts from the public TAT-QA research dataset [20] and plant synthetic traps solely for controlled evaluation. The RQ5 study scans public datasets and documents locally and reports structural signatures and aggregate rates. We did not collect private user documents or intentionally extract or publish personal information that might appear in those public files. Because the tasks were benign financial question answering and summarization, manual review did not expose the research team to harmful or disturbing content.

Provider Feedback We do not frame the measured behavior as a conventional software vulnerability or parser bug: the documents are specification-valid, and the risk arises across OOXML features and ingestion decisions rather than from one faulty component. Nevertheless, before submission we submitted feedback to several tested LLM providers through their available product-feedback or security channels so that they could evaluate their ingestion behavior. These notifications are not used as evidence for our claims, and any acknowledgment, scope decision, or lack of response does not validate or refute the measurements.

Dual Use and Decision to Publish The catalog and fixture generator are functional dual-use artifacts: they can support regression testing, but they could also lower the effort needed to craft a misleading Office document. We mitigate this risk by using only benign factual traps, excluding harmful instructions and executable payloads, reporting provenance-aware defenses, and notifying providers before publication. Publication cannot eliminate misuse risk. We nevertheless decided to conduct and publish the study because closed ingestion behavior cannot be evaluated from offline components alone, the underlying OOXML features are already publicly documented, and a reproducible benchmark enables providers, maintainers, and users to identify and reduce the risk. We judge these defensive benefits to outweigh the residual live-service and dual-use costs under the safeguards above.